

MCP with Agent Framework

Discover and invoke external tools through a controlled client boundary

Source: <https://learn.microsoft.com/en-us/agent-framework/agents/tools/local-mcp-tools?tabs=python>

MCP Base Protocol

- **Messages** — JSON-RPC 2.0 requests, responses, and notifications are the only message shapes the protocol defines
- **Lifecycle** — The initialize → initialized handshake negotiates protocol version and capabilities before normal operation begins
- **Transports** — stdio (client launches the server as a subprocess) and Streamable HTTP (with optional SSE) are the two standard transports
- **Authorization** — Optional OAuth 2.1-based framework for HTTP transports; stdio implementations use environment credentials instead
- **Utilities** — Ping (liveness check), Cancellation (notifications/cancelled), and Progress (progressToken updates) support long-running or unreliable connections

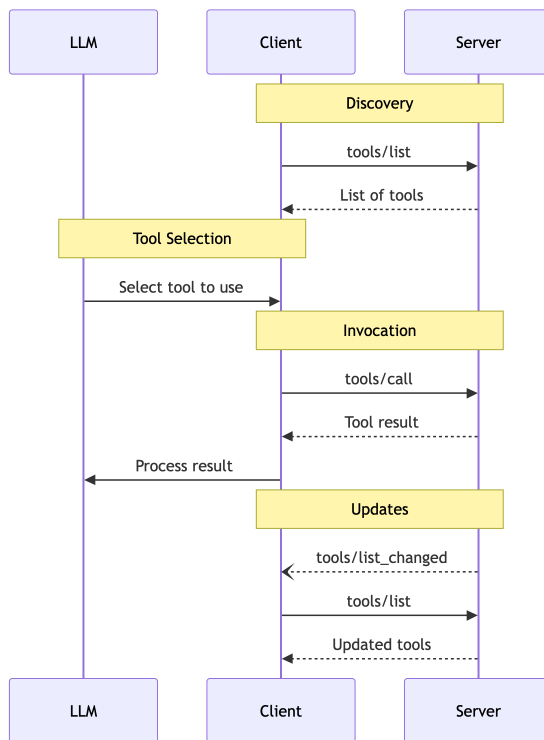
MCP Client Features

- **Roots** — Servers can query which filesystem/URI boundaries a client exposes, so they know where they're allowed to operate
- **Sampling** — Servers can request an LLM completion through the client, keeping model access, prompt content, and approval under client control
- **Elicitation** — Servers can ask the user for additional structured input mid-interaction, validated against a JSON schema

MCP Server Features

- **Tools** — Model-callable functions that let an agent perform actions or invoke external systems — the loop covered next
- **Resources** — Context and data (files, schemas, app-specific info) a user or model can bring into the conversation
- **Prompts** — Reusable, user-triggered templates and workflows a server exposes, e.g. as slash commands
- **Utilities** — Completion (argument autocomplete), Logging (structured log levels), and Pagination (cursor-based paging for large list results)

MCP Tool Message Flow



Pick the transport that matches the server

MAF type	Connection	Use when
MCPStdioTool	Local child process over stdin/stdout	You launch and trust a local server
MCPStreamableHTTPTool	Remote HTTP with streamable responses/SSE	A hosted endpoint exposes MCP over HTTP
MCPWebsocketTool	WebSocket	The server explicitly provides a WebSocket endpoint

Local stdio is a child-process boundary

```
async with MCPStdioTool(  
    name="calculator",  
    command="uvx",  
    args=["mcp-server-calculator"],  
) as mcp:  
    result = await agent.run(  
        "Calculate the total.",  
        tools=mcp,  
    )
```

D E M O

~5 min

Local stdio MCP tool call, end to end

Run the previous slide's exact code live: connect to a local mcp-server-calculator child process, watch the agent discover and invoke a tool through it, and close the connection — client/server loop end to end.

Runbook: [demos/day3/module-5-demo-1-stdio-mcp.md](#)

Remote HTTP belongs behind explicit auth

```
async with MCPStreamableHTTPTool(  
    name="docs",  
    url="https://learn.microsoft.com/api/mcp",  
    allowed_tools=["microsoft_docs_search"],  
) as mcp:  
    result = await agent.run(  
        "Find the current guidance.",  
        tools=mcp,  
    )
```

header_provider has two call shapes

Tool-call request

- Receives `function_invocation_kwargs`
- Can derive request-scoped headers
- Keep secrets outside model-visible arguments

Ambient request

- Receives `{}`
- Includes initialization, discovery, and pings
- Provider must still acquire required connection auth

Reduce the model's tool surface

allowed_tools

Expose an explicit MCP allow-list

Clear names

Avoid collisions by selecting or wrapping distinct tool names

Progressive exposure — EXPERIMENTAL

Add tools for the next loop iteration only

Reset

Progressive tool changes re-arm on each new agent run

approval_mode creates a human boundary

Setting	Effect	Appropriate use
always_require	Every exposed MCP tool needs approval	Unknown or high-risk server
never_require	Calls proceed without approval	Trusted, low-risk read tools
Per-tool mapping	Different rules by tool name	Read automatically; review writes

D E M O

~5 min

approval_mode pauses a write tool for review

Set `approval_mode="always_require"` on the MCP tool and run a request that needs it: the run pauses with a user-input request naming the tool and arguments, and only resumes after an explicit approval decision — the same pause/resume mechanic the official function-tool sample demonstrates, wired onto an MCP tool.

Runbook: [demos/day3/module-5-demo-2-approval-mode.md](#)

Security is broader than authentication

Trust

Review the server operator and package provenance

Least privilege

Narrow identity scopes and allowed_tools

Prompt injection

Treat tool metadata and results as untrusted

Audit

Log selected tool, arguments, approval, result status, and caller

Local or remote?

Question	Prefer local stdio	Prefer remote HTTP
Who hosts it?	Your process or workstation	Trusted service provider
Updates	You pin and manage	Provider manages
Authentication	Local environment / server mechanism	Headers or service identity
Network	No remote endpoint required	Endpoint must be reachable
Operational burden	Higher	Lower

Takeaways

- ✓ You let the server transport determine the MAF MCP class.
- ✓ You constrain discovery with an explicit allow-list.
- ✓ You design `header_provider` for both tool-call and ambient requests.
- ✓ You require approval for risky operations and inspect arguments.
- ✓ You use async context managers to clean up every connection.